

```

    } else {
        out.println("<h3>Please log in first.</h3>");
        out.println("<a href='login.html'>Login</a>");
    }
}
}

```

● LogoutServlet.java

```

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class LogoutServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        if (session != null) {
            session.invalidate();
        }

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<h2>You have been successfully logged out.</h2>");
        out.println("<a href='login.html'>Login Again</a>");
    }
}

```

41. Create a servlet that counts how many times a user has visited the page during a session. Display the visit count on each load.

● VisitCountServlet.java

```

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class VisitCountServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        HttpSession session = request.getSession();

        // Retrieve the visit count from session
        Integer visitCount = (Integer) session.getAttribute("visitCount");

        if (visitCount == null) {
            visitCount = 1;
            out.println("<h2>Welcome! This is your first visit in this session.</h2>");
        } else {
            visitCount++;
            out.println("<h2>Welcome back! You have visited this page " + visitCount +
                " times in this session.</h2>");
        }
    }
}

```

```

        // Store the updated count back in session
        session.setAttribute("visitCount", visitCount);
    }
}

```

42. Develop a servlet that allows the user to select a background color from a dropdown list. Save the selected color in the session and apply it to the response page.

```

colorselect.html
<form action="ColorSessionServlet" method="post">
    Select Background Color:
    <select name="bgcolor">
        <option value="white">White</option>
        <option value="lightblue">Light Blue</option>
        <option value="lightgreen">Light Green</option>
        <option value="yellow">Yellow</option>
        <option value="pink">Pink</option>
    </select>
    <input type="submit" value="Set Background">
</form>

```

ColorSessionServlet.java

```

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class ColorSessionServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String selectedColor = request.getParameter("bgcolor");

        HttpSession session = request.getSession();
        session.setAttribute("bgcolor", selectedColor);

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<body style='background-color:" + selectedColor + ";>");
        out.println("<h2>Background color set to " + selectedColor + "</h2>");
        out.println("</body>");
    }

    // Optional: Allow refreshing using GET
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        String color = "white";

        if (session != null && session.getAttribute("bgcolor") != null) {
            color = (String) session.getAttribute("bgcolor");
        }

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<body style='background-color:" + color + ";>");
        out.println("<h2>Welcome! Your selected background color is " + color +
".</h2>");
        out.println("</body>");
    }
}

```

```
}  
}
```

43. Write a servlet program that checks if a user has logged in. If not, redirect to a login form. Once logged in, store the username in the session and display a welcome message.

logincheck.html

```
<form action="LoginCheckServlet" method="post">  
    Enter Username: <input type="text" name="username"><br>  
    <input type="submit" value="Login">  
</form>
```

LoginCheckServlet.java

```
import java.io.*;  
import javax.servlet.*;  
import javax.servlet.http.*;  
  
public class LoginCheckServlet extends HttpServlet {  
    protected void doPost(HttpServletRequest request, HttpServletResponse response)  
        throws ServletException, IOException {  
  
        String username = request.getParameter("username");  
  
        if (username != null && !username.trim().isEmpty()) {  
            HttpSession session = request.getSession();  
            session.setAttribute("username", username);  
            response.sendRedirect("WelcomeCheckServlet");  
        } else {  
            response.setContentType("text/html");  
            PrintWriter out = response.getWriter();  
            out.println("<h3>Username cannot be empty!</h3>");  
            out.println("<a href='logincheck.html'>Try Again</a>");  
        }  
    }  
}
```

WelcomeCheckServlet.java

```
import java.io.*;  
import javax.servlet.*;  
import javax.servlet.http.*;  
  
public class WelcomeCheckServlet extends HttpServlet {  
    protected void doGet(HttpServletRequest request, HttpServletResponse response)  
        throws ServletException, IOException {  
  
        HttpSession session = request.getSession(false);  
        response.setContentType("text/html");  
        PrintWriter out = response.getWriter();  
  
        if (session != null && session.getAttribute("username") != null) {  
            String username = (String) session.getAttribute("username");  
            out.println("<h2>Welcome, " + username + "!</h2>");  
        } else {  
            // Redirect if not logged in  
        }  
    }  
}
```

```

        response.sendRedirect("logincheck.html");
    }
}
}

```

44. Write a JSP to print Hello World.

 **hello.jsp**

```

<html>
<head><title>Hello JSP</title></head>
<body>
    <h2>Hello World from JSP</h2>
</body>
</html>

```

45. Write a JSP that accepts Student's details from the HTML page and displays all the information.

 **studentform.html**

```

<form action="studentdetails.jsp" method="post">
    Name: <input type="text" name="name"><br>
    Roll No: <input type="text" name="rollno"><br>
    Course: <input type="text" name="course"><br>
    Semester: <input type="text" name="semester"><br>
    <input type="submit" value="Submit">
</form>

```

 **studentdetails.jsp**

```

<html>
<head><title>Student Details</title></head>
<body>
    <h2>Student Information</h2>
    Name: <%= request.getParameter("name") %><br>
    Roll No: <%= request.getParameter("rollno") %><br>
    Course: <%= request.getParameter("course") %><br>
    Semester: <%= request.getParameter("semester") %><br>
</body>
</html>

```

46. Write a JSP that accepts a year from the user and prints whether it is a leap year or not.

 **yearform.html**

```

<form action="leapyear.jsp" method="post">
    Enter a year: <input type="text" name="year"><br>
    <input type="submit" value="Check Leap Year">
</form>

```

 **leapyear.jsp**

```

<%@ page import="java.io.*" %>
<%
    String yearStr = request.getParameter("year");
    int year = 0;
    boolean isLeap = false;

    if (yearStr != null && !yearStr.isEmpty()) {
        try {
            year = Integer.parseInt(yearStr);
            if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0) {

```

```

        isLeap = true;
    }
    } catch (NumberFormatException e) {
        out.println("<h3>Please enter a valid year.</h3>");
    }
}
%>
<html>
<body>
<% if (year > 0) { %>
    <h2><%= year %> is <%= isLeap ? "a leap year." : "not a leap year." %></h2>
<% } %>
</body>
</html>

```

47. Write a JSP program to calculate simple interest by accepting values from the user.

interestform.html

```

<form action="simpleinterest.jsp" method="post">
    Principal: <input type="text" name="principal"><br>
    Rate of Interest: <input type="text" name="rate"><br>
    Time (in years): <input type="text" name="time"><br>
    <input type="submit" value="Calculate">
</form>

```

simpleinterest.jsp

```

<%@ page import="java.text.DecimalFormat" %>
<%
    String p = request.getParameter("principal");
    String r = request.getParameter("rate");
    String t = request.getParameter("time");

    double principal = 0, rate = 0, time = 0, si = 0;
    boolean valid = false;

    try {
        if (p != null && r != null && t != null) {
            principal = Double.parseDouble(p);
            rate = Double.parseDouble(r);
            time = Double.parseDouble(t);
            si = (principal * rate * time) / 100;
            valid = true;
        }
    } catch (Exception e) {
        valid = false;
    }

    DecimalFormat df = new DecimalFormat("0.00");
%>
<html>
<body>
<% if (valid) { %>
    <h2>Simple Interest: <%= df.format(si) %></h2>
<% } else { %>
    <h3>Please enter valid numeric inputs.</h3>
<% } %>
</body>
</html>

```

48. Write a JSP page that performs a division operation. If the user enters zero as the divisor, handle the exception using a custom error page. Use the `isErrorPage="true"` attribute in the error page to display the exception details.

 **divisionform.html**

```
<form action="division.jsp" method="post">
    Numerator: <input type="text" name="num"><br>
    Denominator: <input type="text" name="den"><br>
    <input type="submit" value="Divide">
</form>
```

 **division.jsp**

```
<%@ page errorPage="errorpage.jsp" %>
<%
    int a = Integer.parseInt(request.getParameter("num"));
    int b = Integer.parseInt(request.getParameter("den"));
    int result = a / b;
%>
<html>
<body>
    <h2>Result: <%= result %></h2>
</body>
</html>
```

 **errorpage.jsp**

```
<%@ page isErrorPage="true" %>
<html>
<body>
    <h2 style="color:red;">An error occurred:</h2>
    <p><%= exception.getMessage() %></p>
</body>
</html>
```

49. Configure an error page in web.xml that catches HTTP 404 errors (page not found). Display a user-friendly message using a JSP error page without exposing technical details.

 **error404.jsp**

```
<html>
<head><title>Page Not Found</title></head>
<body style="font-family: Arial; color: darkred;">
    <h2>404 - Page Not Found</h2>
    <p>The page you're looking for doesn't exist or has been moved.</p>
    <p><a href="index.html">Go to Home</a></p>
</body>
</html>
```

 **web.xml (include this inside <web-app>)**

```
<error-page>
    <error-code>404</error-code>
    <location>/error404.jsp</location>
</error-page>
```

50. Write a JSP application that uses `<jsp:include>` to include reusable pages such as header and footer into a main content page.

 **header.jsp**

```
<div style="background-color: lightgray; padding: 10px;">
```

```

    <h2>My Website Header</h2>
    <hr>
</div>

```

 **footer.jsp**

```

<hr>
<div style="background-color: lightgray; padding: 10px;">
    <p>&copy; 2025 My Website. All rights reserved.</p>
</div>

```

 **main.jsp**

```

<%@ page language="java" contentType="text/html; charset=UTF-8" %>
<html>
<head><title>Main Page</title></head>
<body>

<jsp:include page="header.jsp" />

<h3>Welcome to the main content area!</h3>
<p>This is the body of the web page.</p>

<jsp:include page="footer.jsp" />

</body>
</html>

```

51. Create a JSP application where the user enters a favorite color; use <jsp:forward> in process.jsp to redirect to blue.jsp if the input is "blue", otherwise forward to default.jsp — ensure no output is sent before forwarding.

 **colorinput.html**

```

<form action="process.jsp" method="post">
    Enter your favorite color: <input type="text" name="color"><br>
    <input type="submit" value="Submit">
</form>

```

 **process.jsp**

```

<%
    String color = request.getParameter("color");

    if ("blue".equalsIgnoreCase(color)) {
        RequestDispatcher rd = request.getRequestDispatcher("blue.jsp");
        rd.forward(request, response);
    } else {
        RequestDispatcher rd = request.getRequestDispatcher("default.jsp");
        rd.forward(request, response);
    }
%>

```

 **blue.jsp**

```

<html>
<body style="background-color: blue; color: white;">
    <h2>You selected blue as your favorite color!</h2>
</body>
</html>

```

 **default.jsp**

```

<html>

```

```
<body>
  <h2>You did not select blue. Enjoy your custom page!</h2>
</body>
</html>
```

52. Create a JSP page that uses the <jsp:useBean> tag to instantiate a User JavaBean with properties name, email, and age, and display the user details on the page.

User.java (JavaBean)

```
public class User {
    private String name;
    private String email;
    private int age;

    public User() {}

    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }

    public int getAge() {
        return age;
    }
    public void setAge(int age) {
        this.age = age;
    }
}
```

userInput.html

```
<form action="displayuser.jsp" method="post">
  Name: <input type="text" name="name"><br>
  Email: <input type="email" name="email"><br>
  Age: <input type="text" name="age"><br>
  <input type="submit" value="Submit">
</form>
```

displayuser.jsp

```
<jsp:useBean id="user" class="User" scope="request" />
<jsp:setProperty name="user" property="*" />

<html>
<head><title>User Info</title></head>
<body>
  <h2>User Details</h2>
  Name: <%= user.getName() %><br>
  Email: <%= user.getEmail() %><br>
  Age: <%= user.getAge() %>
</body>
</html>
```

53. Create a JavaBean called EmployeeBean with properties for name, designation, and salary. Use the <jsp:useBean> tag to instantiate the bean with the ID employee and set its scope to session. Then, set the name, designation, and salary properties using <jsp:setProperty> tags. Finally, retrieve and display these properties using expression language \${}.

 EmployeeBean.java

```
public class EmployeeBean {
    private String name;
    private String designation;
    private double salary;

    public EmployeeBean() {}

    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }

    public String getDesignation() {
        return designation;
    }
    public void setDesignation(String designation) {
        this.designation = designation;
    }

    public double getSalary() {
        return salary;
    }
    public void setSalary(double salary) {
        this.salary = salary;
    }
}
```

 employeeform.html

```
<form action="employee.jsp" method="post">
    Name: <input type="text" name="name"><br>
    Designation: <input type="text" name="designation"><br>
    Salary: <input type="text" name="salary"><br>
    <input type="submit" value="Submit">
</form>
```

 employee.jsp

```
<jsp:useBean id="employee" class="EmployeeBean" scope="session" />
<jsp:setProperty name="employee" property="*" />

<html>
<body>
    <h2>Employee Information</h2>
    Name: ${employee.name} <br>
    Designation: ${employee.designation} <br>
    Salary: ${employee.salary}
</body>
</html>
```

54. Write a JSP program to demonstrate the use of application implicit object. On the first JSP page, use the application object to maintain a global visitor counter that increments every time the page is accessed. Display the total number of users who have visited the site since the server started.

 **visitor.jsp**

```
<%
    Integer counter = (Integer) application.getAttribute("counter");
    if (counter == null) {
        counter = 1;
    } else {
        counter++;
    }
    application.setAttribute("counter", counter);
%>

<html>
<head><title>Visitor Counter</title></head>
<body>
    <h2>Welcome to the site!</h2>
    <p>Total Visitors Since Server Start: <%= application.getAttribute("counter")
%></p>
</body>
</html>
```

